

SEPARAÇÃO: UNIVERSAL vs. ESPECÍFICO (v2 — sem tabelas)

Data: 2026-08-23

Versão: 2 (listas em vez de tabelas — PDF-safe)

Contexto: Clarificar quais componentes da Fábrica são replicáveis universalmente e quais são específicos do domínio de publicações.

Objetivo: Evitar cópia cega. Novo projeto copia APENAS universal, adapta ESPECÍFICO.

⚡ REGRA DE OURO

UNIVERSAL (copiar 100%): - Estrutura e padrões (agnóstico de domínio) - Infraestrutura (junctions, hooks, CI/CD) - Governança (CLAUDE.md R0–R17) - Economia de tokens (caveman, headroom, lean-ctx, RTK)

ESPECÍFICO (usar como template, reescrever 100%): - Skills de redação/especialização - Comandos de criação (workflow de domínio) - Gates de validação (critérios de domínio) - Templates de output (formato final) - Framework de escrita (seções/estrutura)

UNIVERSAL — COPIAR INTEGRALMENTE

Governança & Padrões

CLAUDE.md (Seção 0–7): - Seção 0: Economia Severa de Tokens → copiar 100% - Seção 1: Regras Globais (R1–R17) → copiar ~80% (adaptar exemplos) - Seção 2: Squad → copiar padrão, reescrever nomes - Seção 3–7: Estrutura → copiar 100%

RTK.md: - Guia completo de token economy - Copiar 100% (sem adaptações)

settings.json: - Hooks PostToolUse e SessionStart - Copiar 100% (adaptar paths)

Padrão de Skills: - Frontmatter (name, description) - Estrutura de fluxo (Passo 1, 2, 3...) - Checklist de entrega - Copiar 100% (mudar conteúdo apenas)

5 Skills de Economia

1. **caveman** — Pensamento telegráfico (3–5 linhas)
2. **headroom** — Comprimir logs >7 linhas (3+4)
3. **lean-ctx** — Grep antes de read, offset/limit
4. **rtk-memory** — Rastreamento de economia por sessão
5. **pre-flight-check** — Validação pré-launch, evita loops

Copiar pastas inteiras (.claude/skills/caveman, etc.). Não adaptar.

Padrões Técnicos

Padrão de Comando: - Fases: P&D → Manufatura → Review → Entrega - Checklist de requisitos contratuais - Passo-a-passo estruturado - Copiar 100%

Padrão de Subagentes: - Atomização de tarefas - Pool com lotes (4 paralelos) - Retentativa 3x com backoff exponencial - Barreira (aguarde todos antes do próximo lote) - Escalação para revisor se falhas persistem - Copiar 100%

Padrão de Script Python: - Docstring com entrada/saída/garantias - Função console_utf8() (UTF-8 everywhere) - Estrutura: try → relatorio[“status”] → finally → JSON - Exit code 0 = sucesso, >0 = falha - Idempotência (rodar 2x = mesmo resultado) - Copiar 100%

Padrão de MCP: - Registro declarativo em .mcp.json - Função por MCP (db_state, file_writer, pdf_gen, code-review-graph) - Copiar 100% (adaptar paths)

Infraestrutura

Portabilidade Multi-IDE: - setup-links.ps1 (Windows) - setup-links.sh (Mac/Linux) - Junctions: agentic/ → .claude/ - Hardlinks: CLAUDE.md - Copiar 100%

Pre-commit Hook: - Validação Python syntax - Auditoria de gates (se output/ modificado) - Checklist CLAUDE.md + .mcp.json - Copiar 100%

Auto-memória: - MEMORY.md (índice de memórias) - RTK-SCRATCHPAD.md (aprendizados de sessão) - Padrão: frontmatter + seções numeradas - Copiar 100%

Code-review-graph: - MCP de inteligência estrutural - PostToolUse hook para atualização automática - Copiar 100% (instalar via uvx)

ESPECÍFICO — USAR COMO TEMPLATE, REESCREVER 100%

Skills de Redação/Especialização

Fábrica (exemplos): - pesquisador: varredura web + dossiê RAG - arquiteto: sumário macro + planejamento - estrategista: draft de capítulo - redator-eita: texto final (7 seções EITA-V2) - revisor-tecnico: peer review - compilador-abnt: merge + formatação ABNT

Para novo domínio: - Identificar 3–5 especialistas chave - Criar skill para cada (usar padrão universal) - Reescrever lógica 100% (mantém frontmatter + fluxo)

Exemplo SaaS: - product-manager: visão + requisitos → feature spec - architect: design → decisões arquiteturais - dev-senior: implementação → código testado - qa-engineer: testes → cobertura ≥90% - (mantém caveman, headroom, lean-ctx para economia)

Comandos de Criação

Fábrica: - /criar-livro: tema → P&D → lotes → compilação - /criar-tcc: estrutura acadêmica → fases - /criar-artigo: compressão de livro - /criar-playbook: extração de cards - /compilar-mega-livro: orquestração final

Para novo domínio: - /criar-feature: epic → design → implementação - /debugar-issue: triage → fix → validation - /refactor-seguro: teste-driven refactor - /deploy-seguro: staging → produção - (mantém padrão de fases + checklist)

Reutilizar: - Padrão de orquestração (fases, barreiras, retentativa) - Estrutura de checklist (requisitos contratuais) - Lógica de escalação (defeitos → revisor)

Gates de Validação

Fábrica (exemplos): - validar-referencias: URL/DOI reais (reprova 4xx/DNS) - validar-metricas: ≥1 métrica por capítulo com valor+unidade - validar-escala: contorno em “Aplica” existe - validar-afirmacoes: dado factual tem citação [N] - validar-fontes: ≥70% referências classe A+B - validar-codigo: smoke test de python/js/bash

Para novo domínio: - validar-testes: cobertura ≥90% - validar-performance: P99 latency < threshold - validar-seguranca: OWASP top 10 check - validar-compliance: legal/regulatory requirements - validar-accessibility: a11y scores

Reutilizar: - Padrão de script Python (console_utf8, JSON report, exit code) - Estrutura de encadeamento (auditar-obra.py agrupa gates) - Relatório de validação (lista de erros/alertas)

Templates de Output

Fábrica: - template_eita.md: 7 seções estruturadas - template.typ: ABNT (capa, folha de rosto, CIP, sumário) - template_lead_magnet.html: A4 + CTA

Para novo domínio: - Seu template estruturado (N seções) - Seu formato final (PDF, HTML, Markdown, PPTX) - Seu CTA/call-to-action

Reutilizar: - Padrão de gerador de esqueleto (gerar-esqueleto-*.py) - Padrão de compilação (Pandoc → Typst ou seu motor) - Metadados (frontmatter YAML para valores variáveis)

Framework de Escrita

Fábrica: - EITA-V2: 7 seções (Intro, Explica, Ilustra, Técnica, Aplica, Conclusão, Refs) - Cada seção tem regras de tom, comprimento, citações - Gates validam estrutura (headers numerados, seções completas)

Para novo domínio: - Seu framework (quantas seções? Quais nomes?) - Suas regras por seção (tom, comprimento, links obrigatórios?) - Seus gates (validar headers, seções, citações)

Reutilizar: - Padrão de parser (regex para headers, splitBatches) - Padrão de validação (gerar-esqueleto → gates → revisão) - Padrão de refinamento (reescrever → gates → compilação)

Compilação Final

Fábrica: - Input: Markdown (7 seções, referências ABNT) - Processo: Pandoc → Typst → typst compile - Output: PDF (ABNT formatado)

Para novo domínio: - Input: seu formato (Markdown, Jupyter, Word, XML) - Processo: seu pipeline (seu motor de conversão) - Output: seu formato final (PDF, HTML, EPUB, PPTX)

Reutilizar: - Padrão de compilador (entrada → processamento → output) - Padrão de erro handling (relatorio JSON com sucesso/erros) - Padrão de idempotência (rodar 2x = mesmo resultado)

MCPs Específicas

Fábrica: - db_state: SQLite com tabelas (obras, capitulos, status) - Queries: listar fases, buscar por slug, atualizar status - Uso: registrar progresso, auditoria, rastreamento

Para novo domínio: - db_state: SQLite com suas tabelas (features, sprints, deployments) - Queries: suas entidades + atributos - Uso: seu contexto operacional

Reutilizar: - Padrão de MCP (comando node, args com path db) - Padrão de queries (SELECT/INSERT/UPDATE com relatorio JSON) - Integração em settings.json

Squad Especializada

Fábrica: - F1 (Pesquisa): pesquisador → arquiteto (30 min) - F2 (Manufatura): estrategista → redator-eita (4h, lotes 4) - F2.5 (Review): auditar-obra → revisor-tecnico (1h) - F3 (Entrega): compilador-abnt → PDF (30 min) - F4 (Coleção): derivadas paralelas (2h)

Para novo domínio: - Sua estrutura de fases (P&D? Design? Implementation? QA? Deployment?) - Seus especialistas por fase (quem faz o quê?) - Seus tempos estimados - Suas métricas de sucesso

Reutilizar: - Padrão de fases (estruturada, com barreiras) - Padrão de pool (lotes paralelos) - Padrão de retentativa (3x com escalação) - Padrão de relatório (tempo + status + próximos passos)

EXEMPLOS PRÁTICOS: COPIAR vs. REESCREVER

Exemplo 1: Padrão de Skill (COPIAR)

Universal (copiar 100%):

name: nome-skill

description: Uma frase do resultado

```
# Skill_NomeCapitalizado
```

Você é o especialista em [DOMÍNIO].

```
## Regras
```

```
- REGRA 1, 2, 4 do CLAUDE.md
```

```
## Padrão de Fluxo
```

```
### Passo 1 – [Sua fase 1]
```

```
### Passo 2 – [Sua fase 2]
```

```
### Passo 3 – [Sua fase 3]
```

```
## Checklist de Entrega
```

```
- [ ] Artefato gerado
```

```
- [ ] Tests passando
```

```
- [ ] Commit feito
```

✅ Copiar frontmatter + estrutura. Reescrever apenas [DOMÍNIO], [fases], [checklist].

Exemplo 2: Skill de Domínio (REESCREVER)

Fábrica (redator-eita):

Você é redator de manufatura final (EITA-V2).

```
## Seções obrigatórias
```

```
1. Introdução (acessível)
```

```
2. Explica (teoria)
```

```
3. Ilustra (analogia + diagrama)
```

```
4. Técnica (código + arquitetura, 60%)
```

```
5. Aplica (caso real, contorno)
```

```
6. Conclusão (síntese)
```

```
7. Referências (ABNT)
```

❌ Não copie. Reescreva para seu domínio:

SaaS (feature-implementer):

Você é implementador de features (Sprint).

```
## Seções obrigatórias
```

```
1. Context (por que existe, problem statement)
```

```
2. Design (arquitetura, decisões)
```

```
3. Implementation (código, padrões, 70%)
```

```
4. Tests (cobertura ≥90%, happy path + edge)
```

```
5. Deployment (staging → prod, rollback)
```

```
6. Monitoring (métricas, alertas)
```

```
7. Docs (README, API, runbook)
```

Exemplo 3: Script Determinístico (COPIAR PADRÃO)

Padrão universal:

```
#!/usr/bin/env python3
```

```
"""
```

```
<modulo>: <descricao>
```

```
Entrada: <seu input>
```

```
Saída: output/<slug>/relatorio-<modulo>.json
```

Garantias:

- Idempotente (rodar 2x = mesmo resultado)
 - Sem side effects fora de output/
 - Relatório JSON sempre gravado
 - Exit code 0 = sucesso, >0 = falha
- """

```
import argparse, json, sys
from pathlib import Path

def console_utf8():
    for fluxo in (sys.stdout, sys.stderr):
        try:
            fluxo.reconfigure(encoding="utf-8", errors="replace")
        except (AttributeError, OSError):
            pass

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("slug", help="slug-da-obra")
    parser.add_argument("--opcao", default="valor")
    args = parser.parse_args()

    relatorio = {"status": "falha", "erros": []}

    try:
        # ← SUA LÓGICA AQUI
        relatorio["status"] = "sucesso"
    except Exception as e:
        relatorio["erros"].append(str(e))
        return 1
    finally:
        Path(f"output/{args.slug}/relatorio-meu-script.json").write_text(
            json.dumps(relatorio, indent=2, ensure_ascii=False)
        )

    return 0 if relatorio["status"] == "sucesso" else 1

if __name__ == "__main__":
    console_utf8()
    sys.exit(main())
```

✓ Copiar estrutura completa. Reescrever apenas [SUA LÓGICA].

BLUEPRINT DE REPLICAÇÃO — PASSO A PASSO

Fase 1: Clonar + Remover Específico (2.5 horas)

Passo 1a: Copiar tudo

```
cp -r proj_fabrica-de-livros novo-projeto
cd novo-projeto
```

Passo 1b: Remover específico da Fábrica

Deletar skills de redação: - .claude/skills/pesquisador - .claude/skills/arquiteto - .claude/skills/estrategista - .claude/skills/redator-eita - .claude/skills/redator-academico - .claude/skills/revisor-tecnico - .claude/skills/compilador-*

Deletar comandos de criação: - .claude/commands/criar-livro.md - .claude/commands/criar-tcc.md - .claude/commands/criar-artigo.md - .claude/commands/criar-playbook.md - .claude/commands/criar-lead-magnet.md - .claude/commands/compilar-*.md - .claude/commands/reescrever*.md

Deletar scripts específicos: - scripts/validar-referenc*.py - scripts/validar-metricas.py - scripts/validar-escala.py - scripts/validar-afirmacoes.py - scripts/validar-fontes.py - scripts/validar-comando*.py - scripts/gerar-cap*.py - scripts/gerar-epub.py - scripts/compilar-artigo.py - scripts/auditar-obra.py - scripts/tipos_obra.py (será reescrito)

Deletar templates específicos: - templates/template_eita.md - templates/template.typ - templates/template_tcc.typ - templates/template_playbook.typ

Deletar skills específicas de redação: - .claude/skills/compilador-abnt - .claude/skills/compilador-artigo - .claude/skills/compilador-mega-livro

Passo 1c: Validar o que restou (universal)

Manter (✅ universal): - .claude/CLAUDE.md (governança) - .claude/settings.json (hooks) - .claude/RTK.md (economia) - .claude/skills/caveman (economia) - .claude/skills/headroom (economia) - .claude/skills/lean-ctx (economia) - .claude/skills/rtk-memory (economia) - .claude/skills/pre-flight-check (economia) - setup-links.ps1 - setup-links.sh - scripts/pool-capitulos.py (padrão de pool — reescrever nomes) - scripts/pdf_typst.py (compilação genérica) - .git/hooks/pre-commit - .mcp.json (padrão MCP) - data/estado_fabrica.db (schema — reescrever tabelas)

Fase 2: Customizar Universal (1 hora)

Passo 2a: CLAUDE.md - Linha 6: trocar “Fábrica Agêntica de Publicações” → seu tema - Seção 2 (Squad): trocar skills específicas de redação - Manter: Seção 0 (Economia), R1–R17, padrão de fases

Passo 2b: .mcp.json

```
sed -i 's|proj_fabrica-de-livros|novo-projeto|g' .mcp.json
```

Passo 2c: setup-links

```
# Windows
.\scripts\setup-links.ps1 -Verbose
```

```
# Mac/Linux
./scripts/setup-links.sh
```

Fase 3: Reescrever Específico (4–8 horas)

Criar skills de domínio: - Copiar padrão de .claude/skills/caveman - Criar .claude/skills/seu-especialista-1.md - Reescrever lógica 100% (mantém frontmatter + Passo 1/2/3)

Criar comandos de criação: - Copiar padrão de .claude/commands/ (estrutura de fases) - Criar .claude/commands/seu-comando-1.md - Reescrever fluxo 100% (mantém checklist de requisitos)

Criar gates de validação: - Copiar padrão de script Python - Criar scripts/validar-seu-criterio.py - Reescrever lógica 100% (mantém console_utf8, JSON, exit code)

Criar tipos_obra.py:

```

TIPOS = {
    "seu-tipo-1": {
        "rotulo": "Seu Tipo 1",
        "raiz_output": "seu-tipo-1s",
        "natureza": "geracao", # ou compressao, extracao
        "custo_llm": "medio", # ou alto, baixo, zero
        "gates_conteudo": ["validar-seu-criterio"],
    },
    "seu-tipo-2": {...},
}

```

Criar templates:

```

templates/
├─ template_seu-tipo.md          (sua estrutura)
├─ template_seu-tipo.typ          (seu formato PDF)
└─ template_seu-tipo.html         (seu formato web)

```

Atualizar db_state (SQLite): - Editar schema (tabelas próprias: seu-tipos, sprints, etc.) - Manter padrão (CREATE TABLE, columns, queries)

Fase 4: Validar (1 hora)

```

# Python syntax
python -m py_compile scripts/*.py
echo "✓ Python OK"

# CLAUDE.md carregado
grep "alwaysApply: true" CLAUDE.md
echo "✓ Governança OK"

# MCPs
cat .mcp.json | grep -c "mcpServers"
echo "✓ MCPs OK"

# Skills universal
test -d .claude/skills/caveman && echo "✓ Economia OK"

# Pre-commit hook
test -f .git/hooks/pre-commit && echo "✓ Hook OK"

# Setup links
test -L agentic && echo "✓ Junctions OK"

```

TEMPOS ESTIMADOS (REALISTA)

Fase 1 (Clonar + remover): **2.5 horas** - Clonar: 10 min - Remover específico: 1.5h - Validar universal: 30 min - Git commit: 10 min

Fase 2 (Customizar universal): **1 hora** - CLAUDE.md: 20 min - .mcp.json: 10 min - setup-links: 10 min - Git commit: 10 min

Fase 3 (Reescrever específico): **4–8 horas** - Skills domínio: 1–2h - Comandos criação: 1–2h - Gates validação: 1–2h - Templates: 30 min - tipos_obra.py: 30 min - db_state schema: 30 min

Fase 4 (Validar): **1 hora** - Testes: 30 min - Ajustes: 20 min - Git commit + push: 10 min

TOTAL: 8.5–12.5 horas (vs. 4–6 semanas do zero)

CHECKLIST FINAL

UNIVERSAL (copiar 100%): - ☐ CLAUDE.md Seção 0-7 - ☐ RTK.md - ☐ 5 Skills economia - ☐ settings.json - ☐ setup-links.ps1/.sh - ☐ padrão de scripts Python - ☐ padrão de MCPs - ☐ pre-commit hook - ☐ MEMORY.md framework

ESPECÍFICO (reescrever 100%): - ☐ Skills domínio (3-5) - ☐ Comandos criação (2-5) - ☐ Gates validação (3-6) - ☐ Templates output (1-3) - ☐ tipos_obra.py - ☐ db_state schema - ☐ Squad especializada

TESTAR: - ☐ Python syntax - ☐ CLAUDE.md alwaysApply - ☐ MCPs inicializam - ☐ Junctions funcionam - ☐ Pre-commit hook bloqueia erros - ☐ Skills listadas em /help

CONCLUSÃO

Copiar 40%: Infraestrutura, padrões, governança, economia (universal)

Reescrever 60%: Skills, comandos, gates, templates, compilação (domínio)

Resultado: Novo projeto com **infraestrutura profissional 100% pronta, 100% customizado para seu escopo**, em **10-14 horas**.

Sem replicação: **4-6 semanas** (do zero)

Com replicação: **10-14 horas** (copiar + reescrever específico)

Ganho: 2-3 semanas de trabalho, infraestrutura pronta, economia de tokens ativa, portabilidade multi-IDE garantida.